

Inhaltsverzeichnis

1. Java Generics	1
1.1. Bounded Generics	2
1.2. Wildcards	3
1.3. Demonstrationen	4
1.4. Übungen	4
1.5. Tipps, Patterns & Best Practices	5

1. Java Generics

Generische Programmierung in Java ist durch **Generics** seit langem möglich. Der Begriff steht synonym für "parametrisierte Typen". Die Idee ist, zusätzliche Variablen für Typen einzuführen. Diese Typ-Variablen repräsentieren zum Zeitpunkt der Implementierung unbekannte Typen. Dazu wird der sogenannte **Diamond-Operator** `<>` bei Klasse oder Methode genutzt:

```
List<T> list; ①  
public Printer<T> { ...}
```

① T oft als Kürzel für **Type**

Erst bei der Verwendung der Klassen, Schnittstellen und Methoden im Code werden diese Typ-Variablen durch konkrete Typen durch den Entwickler ersetzt.

Damit kann generische und **typsichere Programmierung** gewährleistet werden. In der Regel wird die Codemenge durch Generics reduziert (Prinzip: **DRY** (*Don't Repeat Yourself*)), z.T. wird er allerdings dadurch auch schwerer wartbar und die Lesbarkeit nimmt ab. Die folgenden zwei Varianten finden sich in der Praxis am häufigsten:

- Java Generics bzgl. einer **Klasse**
- Java Generics bzgl. einer **Methode**



Viele Beispiele finden sich, wie wir schon gesehen haben, im Collections Framework, etwa bei den Interfaces `List<T>` oder `Map<K,V>`. Siehe dazu z.B. → [Java 17 Package Documentation für java.util!](#)

Beispiel einer generischen Klasse

```
public class Joiner<T> {  
  
    public String join(Collection<T> collection) {  
        StringBuilder builder = new StringBuilder();  
        Iterator<T> iterator = collection.stream().iterator();  
  
        builder.append("[");  
        while (iterator.hasNext()) {  
            T item = iterator.next();
```

```

        final String formattedItem =
            iterator.hasNext()
                ? String.format("%S, ", item)
                : String.format("%S", item);
        builder.append(formattedItem);
    }
    builder.append("]");

    return builder.toString();
}
}

```

Zeile 1 macht die Klasse generisch, in Zeile 9 wird der unbekannte Typ genutzt.

Die Nutzung einer generischen Klasse, z.B. in einem Test, sieht entsprechend so aus, indem der Typ **T** konkret angegeben wird:

```

Joiner<String> joiner = new Joiner<Joiner>();
// Typ kann bei der Initialisierung auch weggelassen werden
Joiner<String> joiner = new Joiner<>();

```

Auch Methoden können generisch sein, sie werden ähnlich wie Klassen definiert:

Beispiel einer generischen Methode

```

public <T> String print(T data)

```

1.1. Bounded Generics

Oft kommen sogenannte Begrenzungsausdrücke oder **bounded generics** zum Einsatz.

"Upper Bound"

Dabei wird bei der Definition die Oberklasse angegeben, von welcher der **generische Typ erben muss**. Die Blickrichtung ist sozusagen "*nach unten*".

Auf diese Weise wird der ansonsten *beliebige* Typ eingeschränkt, sodass der generische Typ zwar immer noch unbekannt ist, aber nicht von *jedem* Typ sein kann, sondern nur entsprechend der Einschränkung, z.B.

```

public <T extends Number> add(T first, T second) { ... }

```

Hier kann z.B. der genutzte Datentyp ausschließlich ein Zahlen-Datentyp sein, der von **Number** erbt.

Auch die **gegenteilige Blickrichtung** "*nach oben*" kann auch angegeben werden:

"Lower Bound"

Der Begrenzungsausdruck

```
<? super FilledCircle>
```

bedeutet, dass der Typ hier ein Supertyp von **FilledCircle** sein muss, also FilledCircle selbst oder eine Oberklasse von dieser.

Es definiert eine untere Grenze für den erlaubten Typ. Dazu ein Beispiel:

```
public void draw(List<? super FilledCircle> list)
```

List<? super FilledCircle> ist hier die Angabe dafür, dass die "draw" Methode die Liste mit bestimmten Typen "zeichnen" kann, nämlich deren Elemente "kreisförmig" sind.

Der Unterschied lässt sich grafisch besser verstehen:

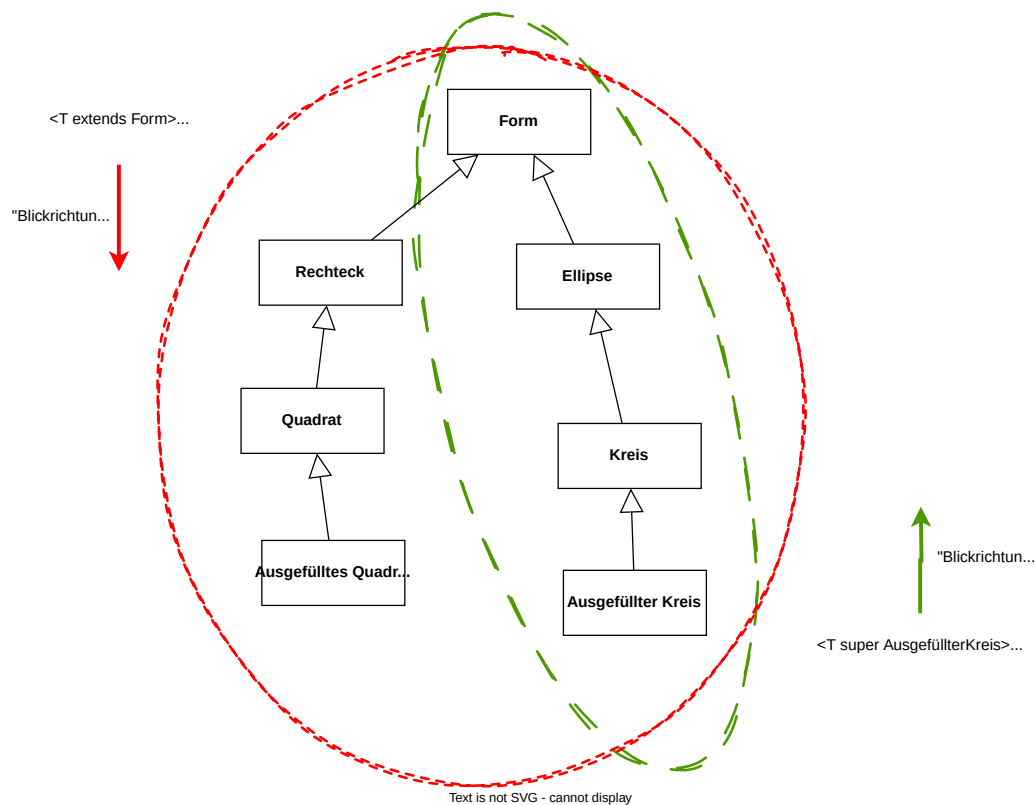


Figure 1. Grafische Erläuterung der Wildcard-Logik

1.2. Wildcards

Im Rahmen der Generics kann man anstelle der Typvariable - oben z.B. **T** - durchaus auch die sogenannte **Wildcard** **?** nutzen. Das schafft Flexibilität hinsichtlich der spezifizierbaren Typen (erschwert aber die Nutzung der Typvariable selbst innerhalb der Klasse, weil **?** für "irgendwas" steht).

1.3. Demonstrationen

Die Unit-Tests zur **Demonstration** finden sich hier:

```
src/test/java/de/dhbw/generics/GenericsDemoTests.java
```

Der zugehörige, in den Tests genutzte **Quellcode** findet sich hier:

```
src/main/java/de/dhbw/generics/demo/*.java
```

1.4. Übungen

Nutze folgendes Package für die **Unit-Tests**:

```
src/test/java/de/dhbw/generics/GenericsExerciseTests.java
```

Die im Test benutzten **Implementierungen** gehören in das Package:

```
src/main/java/de/dhbw/exercises/*.java
```



Zahlentypen in Java haben eine gemeinsame Superklasse `java.lang.Number`.

Übung 1

Ein **Interface**

Erstelle ein **Interface** für einen Taschenrechner, der die vier Grundrechenarten in Form von Methoden zur Verfügung stellt, also für ...

- addieren,
- subtrahieren,
- multiplizieren und
- dividieren.

Der Taschenrechner sollte mit beliebigen Zahlen-Datentypen umgehen können.

Übung 2

Eine **Klasse**

Realisiere einen generischen, aber **konkreten** Taschenrechner für die 4 Grundrechenarten, der alle Zahlen-Datentypen verarbeiten kann. Schreibe dazu hier einen kleinen Test, der die Funktionsfähigkeit mindestens einer der Rechenarten mit Beispielwerten testet, aber mit unterschiedlichen Zahlen-Datentypen.

Übung 3

Implementiere eine konkrete Klasse `Workflow`.

Diese Klasse soll eine statische, generische Methode `execute` bekommen, die beliebige Workflow-Schritte ausführen kann.

Die Workflow-Schritte sollen von einer Eltern-Klasse namens `Step` erben. Implementiere mindestens 2 konkrete Workflow-Schritte.

Realisiere für die "Ausführung" der `execute` Methode einfach eine Konsolenausgabe, z.B. des Names des konkreten Workflow-Schrittes (d.h. der Klassenname).

1.5. Tipps, Patterns & Best Practices

- Bei Listen sollte man immer mittels Diamond-Operator `<>` den Datentyp für die Liste angeben